

Vyapari: A Unified ERP Platform for Retail and Wholesale Businesses of Every Scale

Project Proposal for UCS503P
Submitted to: Miss Paramveer Kaur
Thapar Institute of Engineering and Technology

Vanshika, CSED*

Vanshika, CSED[†]

Nandini Garg, CSED[‡]

August 13, 2026

1 Higher-order goal

... secondary framing

This project aims to help retail and wholesale businesses of every scale – from a single neighbourhood shop to a large multi-outlet chain – run their day-to-day operations online through a single system, instead of juggling separate spreadsheets, ledgers, and point tools. While the long-term vision is a full-fledged Enterprise Resource Planning (ERP) suite, the immediate engineering objective is to translate *unified business operations* into a measurable, deployable web platform.

2 Time-to-value

... primary heuristic

- We will deliver meaningful increments quickly by prototyping the core inventory-to-invoice workflow first and validating it with a mock business within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration (a working sales-and-inventory loop), then improved based on feedback.

3 Problem Statement

Retail and wholesale businesses, regardless of size, typically rely on a patchwork of tools: paper ledgers, spreadsheets, WhatsApp for orders, and separate billing software. Common pain points include:

- **Fragmentation:** inventory, sales, finance, and employee records live in disconnected systems, causing lost context and duplicate data entry.
- **Low visibility:** owners cannot easily see real-time stock levels, cash flow, or sales trends across locations.

*Roll No: 1024030163, Email: vvanshika_be24@thapar.edu

[†]Roll No: 1024030166, Email: vbansal1_be24@thapar.edu

[‡]Roll No: 1024030169, Email: ngarg2_be24@thapar.edu

- **Manual overhead:** invoicing, payroll, and customer follow-ups are handled manually, consuming staff time and inviting errors. Owners also spend time *manually* typing out WhatsApp messages to remind customers or restock suppliers.
- **No forecasting:** businesses restock and staff reactively, since there is no systematic demand or cash-flow forecasting.
- **No churn visibility:** businesses have no systematic way of knowing which customers are likely to stop buying, or which products a given customer is unlikely to repurchase, until sales have already dropped off.

Why this matters now (contextual relevance): As retailers and wholesalers of every scale move online and face thinner margins, even modest automation of inventory, billing, reminders, and reporting can materially reduce lost sales, stockouts, and administrative overhead – and the same platform should scale from a single small shop up to a large multi-branch enterprise without needing to be re-architected.

4 Proposed Solution

... *Software Proposition*

4.1 Overview

Build a **web-based** ERP platform, **Vyapari**, that unifies the core operational and administrative functions a retail or wholesale business needs to run online, at any scale:

- **Inventory:** stock tracking, reorder alerts, multi-location and multi-SKU support.
- **Sales:** order capture, point-of-sale style billing, order status tracking.
- **Finance:** ledgers, expense tracking, profit/loss summaries.
- **CRM:** customer records, purchase history, follow-up reminders.
- **Employees & Payroll:** staff records, attendance, salary computation and disbursement tracking.
- **Invoices:** generation, templating, and GST/tax-ready billing.
- **Analytics:** dashboards for sales, inventory turnover, and financial health.
- **AI chatbot:** conversational assistant for staff/customers to query stock, orders, or FAQs.
- **Forecasting:** lightweight demand and cash-flow forecasting to guide restocking and staffing.
- **Customer churn prediction:** an AI model that scores, for each customer and product, the likelihood of a repeat purchase – flagging customers at risk of churning and products with fading demand for a given customer, so staff can prioritize follow-ups and retention offers.
- **WhatsApp/SMS notifications:** in simple words, the system automatically sends a WhatsApp or SMS message where a staff member would otherwise have typed one by hand – a low-stock alert to the owner, an order-ready or “come back and shop again” nudge to a customer – so the reminders businesses already send informally happen on their own, on time, every time.
- **Barcode/QR scanning:** in simple words, staff point a phone or tablet camera at a product’s barcode or QR code instead of typing its name or code by hand; the app instantly looks it up and adds it to stock or to the current bill, making check-in and checkout faster.

4.2 Core workflow

1. Staff/owner onboards the business: catalog, initial stock, and employee records.
2. Sales are recorded through the POS/sales module – either typed in or scanned via barcode/QR – automatically decrementing inventory and generating an invoice.
3. Finance and payroll modules update ledgers and salary computations from sales, expense, and attendance data.
4. Analytics, forecasting, and churn-prediction modules surface trends, reorder/staffing suggestions, and at-risk customers or products; the chatbot lets staff query the system in natural language; WhatsApp/SMS notifications automatically carry the relevant alerts (e.g., low stock, order ready, “we miss you” offers) to the right person.

4.3 Operational constraints for an educational, course-project setting

- Keep the solution usable on low-to-mid range devices via responsive web design, so it remains accessible whether the business is a single small shop or a large multi-branch operation.
- Avoid dependence on advanced research algorithms; focus on workflow, automation, and system correctness, using off-the-shelf forecasting, churn-prediction, messaging, and scanning components rather than novel models.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach

...Engineering focus

This is an engineering course project, so we focus on scalable module design and integration rather than novel algorithm research.

5.1 Forecasting, chatbot, and churn prediction

...non-research

Forecasting, the chatbot, and churn prediction will be built on mature, off-the-shelf approaches:

- Demand forecasting via a simple moving-average or lightweight time-series model (e.g., exponential smoothing) rather than state-of-the-art forecasting research.
- AI chatbot via a hosted LLM API constrained to a defined set of intents (stock lookup, order status, FAQs) with retrieval over the business’s own data.
- Customer churn prediction via a standard supervised classification model (e.g., logistic regression or gradient-boosted trees) trained on purchase recency, frequency, and monetary (RFM) features per customer–product pair, rather than a novel research model.
- Always show forecasts, churn scores, and chatbot answers as suggestions to staff, never as fully automated, unreviewed actions (e.g., no auto-placed purchase orders without confirmation).

5.2 Notifications and scanning

...simple, off-the-shelf, demoable

These two features are deliberately kept simple to build and easy to explain:

- **WhatsApp/SMS notifications:** sent using an existing, ready-made messaging API (e.g., the WhatsApp Business API or an SMS gateway like Twilio) – the system does not send a message from scratch, it just fills in a template (“Stock of X is low”, “Your order is ready”, “It’s been a while, here’s an offer”) and hands it to the API. Businesses already do this manually today; this module simply automates the trigger and the sending.
- **Barcode/QR scanning:** done using an existing browser-based scanning library (e.g., `html5-qrcode` or `ZXing`) that reads a code through the device camera and returns the product ID – no custom computer-vision model is built. This works with an ordinary phone camera for a small shop, or a dedicated barcode scanner for a larger warehouse.

5.3 Web architecture

...web-based solution

A typical 3-tier, modular web architecture:

- **Frontend:** responsive UI for owners, staff, and customers (dashboards, POS screen with camera-based scan input, catalog/CRM views).
- **Backend API:** authentication and role-based access, module-specific services (inventory, sales, finance, payroll, CRM), and an integration layer for analytics/forecasting/churn-prediction/chatbot/messaging.
- **Database:** relational store for transactional modules (inventory, sales, finance, payroll) plus a document/vector store for chatbot retrieval and a feature store for the churn-prediction model.
- **Third-party integrations:** WhatsApp Business API / SMS gateway for notifications; device camera access for barcode/QR scanning.

5.4 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change across modules.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features (module by module).

6 Evaluation Criterion

...measurable and attributable

We define success using measurable metrics tied to the core sales-to-invoice workflow.

6.1 Primary evaluation metric

Time-to-invoice (TTI): time between a sale being recorded and a corresponding invoice being generated and inventory being updated.

- Target: median TTI ≤ 5 seconds during pilot window.
- Attribution: measured from database timestamps (sale creation vs. invoice/inventory update event).

6.2 Secondary evaluation metrics

- **Stock accuracy:** percentage agreement between system-recorded stock and physical stock counts during pilot audits.
- **Forecast usefulness:** percentage of forecast-driven reorder suggestions accepted by staff.
- **Churn-prediction usefulness:** precision/recall of the churn model against actual repurchase outcomes, and the percentage of flagged at-risk customers who receive and respond to a retention follow-up.
- **Chatbot resolution rate:** percentage of chatbot queries answered correctly without escalation to a human.
- **Notification reliability:** percentage of WhatsApp/SMS alerts successfully delivered, and average delay between trigger event (e.g., low stock) and message sent.
- **Scan-to-bill speed:** average time to add an item to a bill via barcode/QR scan versus manual entry.
- **Reliability:** availability of staging/production for login, billing, and inventory operations during business hours (e.g., $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan

... high-level

- Recruit 2–3 retail/wholesale businesses spanning different scales (a small shop, a mid-sized store, and a larger multi-outlet business), or simulate with mock storefronts, plus 3–5 staff roles.
- Compare metrics before/after within the iteration window by logging baseline estimates via short interviews or existing process observations.

7 Scalability

... with foresight

The proposition should scale in both theoretical and practical senses, and the platform is explicitly designed to serve businesses ranging from a single small outlet to a large enterprise without a change of architecture.

1. **Scalable (theoretically):** The system should support growth in number of SKUs, transactions, and concurrent users by:
 - decoupling modules (inventory/sales/finance/CRM/payroll as independently deployable services),
 - enabling pagination and indexing for common queries (stock lookups, invoice search),
 - using stateless API design for horizontal scaling,
 - queuing outbound WhatsApp/SMS notifications so message volume can grow without blocking the main application.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed database,
 - containerized or platform-hosted deployment per module,
 - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and reactive frontend pages.
- Managed relational database for transactional modules; optional document/vector store for chatbot retrieval.
- Authentication and role-based access approach (token-based or session-based).
- Object storage for product images and invoice PDFs.
- Hosted LLM API for the chatbot module.
- Standard ML library (e.g., scikit-learn) for the churn-prediction model.
- WhatsApp Business API or SMS gateway (e.g., Twilio) for notifications.
- Browser-based barcode/QR scanning library (e.g., `html5-qrcode`, `ZXing`) for camera-based scanning.
- Charting/BI library for the analytics module.
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on module design, integration correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

... iteration-friendly

9.1 Initial deliverable

... first iteration

- Inventory module: add/update/view stock with low-stock alerts
- Sales module: record a sale, auto-generate an invoice, decrement stock
- Basic finance ledger: auto-populated from sales and manual expense entry
- Basic logging and timestamps for TTI measurement
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- CRM module: customer records, purchase history, follow-up reminders
- Employees & Payroll module: attendance and salary computation
- Analytics dashboard: sales, inventory turnover, and financial summaries
- Forecasting module: demand/reorder suggestions
- Customer churn prediction module: per-customer, per-product repurchase likelihood scoring, surfaced as retention suggestions in the CRM

- AI chatbot: stock/order/FAQ query assistant
- WhatsApp/SMS notifications module: automated low-stock alerts to owners and order/reminder messages to customers
- Barcode/QR scanning: camera-based scan-to-add for inventory and scan-to-bill for POS
- Improved search/filtering and indexed queries for scale readiness

10 Risks and mitigations

- **Data privacy / consent:** minimize collected customer and employee data; encrypt sensitive fields (e.g., payroll, payment details); store only what is needed.
- **Staff adoption:** provide an easy UI for billing and stock updates; keep workflow steps minimal; include keyboard-friendly shortcuts for POS use.
- **Forecast/chatbot/churn-model over-trust:** present forecasts, chatbot answers, and churn scores as suggestions requiring staff confirmation, never as fully automated decisions.
- **Notification spam/consent:** only send WhatsApp/SMS messages to customers who have opted in, respect message-frequency limits, and follow applicable messaging regulations.
- **Scanning reliability:** fall back to manual entry when a barcode/QR code is missing, damaged, or unreadable by the camera.
- **Evaluation measurement ambiguity:** instrument timestamps and define clear module boundaries and events before development begins.
- **Operational issues in pilot:** use staging early; ensure CI/CD produces repeatable deployments across all modules.

11 Summary

This project proposes Vyapari, a deployable web-based ERP platform that unifies inventory, sales, finance, CRM, employees/payroll, invoicing, analytics, an AI chatbot, demand forecasting, customer churn prediction, automated WhatsApp/SMS notifications, and barcode/QR scanning for retail and wholesale businesses of every scale, from small shops to large enterprises. It meets the course criteria by emphasizing time-to-value through rapid, module-by-module iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially time-to-invoice). It remains focused on engineering workflow, modular architecture, and scalability readiness rather than research-driven novelty, while demonstrating understanding of large-scale, multi-module system design.